

1. Control Arm Documentation — Version 0.1

The focus of this documentation is outline how to write MoveIt2 Python code, due to the lack of information and documentation present on the internet.

This document contains oversight on how the provided Python script functions and the minimal components for robotic arm movement so that users can building their own scripts from the minimal requirements.

1.1. Python Code

1.1.1. Libraries

1. time
2. `rcply.node`
3. `geometry_msgs.msg`

```
27 import time
26 import rclpy
25 from rclpy.node import Node
24 from geometry_msgs.msg import TwistStamped
```

Figure 1: Python Imported Libraries

The **rcply** (ROS Client Library for Python) is the integrated Python library that allows for communication of the MoveIt2 (ROS2) commands to make auriculate movements of a compatible robotic arm. The library uses pre-defined classes that greatly allow for the arm movements. The library itself is not exclusive to robotic arms but any robotic features.

Accessing the ROS2 integrated libraries users must **"source"** their environments to allow usages of ROS2 and MoveIt2 applications.

1.1.2. Panda_Link0

Due the limitations it was deemed necessary to utilise the pre-defined robotic arm given with the installation of ROS2 and MoveIt2. The **Panda_Link0** similar at the desired robotic arm for the project in its articulation points.

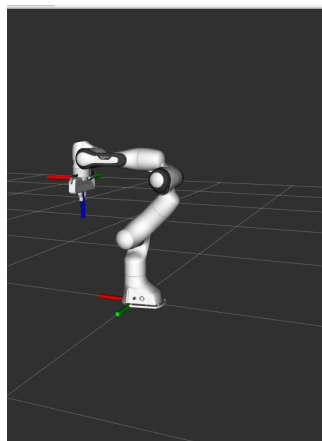


Figure 2: Panda Link Robotic Arm

1.1.3. Node

The *class ServoNode(Node)* inheres the classes within rclpy. The functions *def __init__* and *def send* initiate the class and allow for "messages" to be sent from the node to the robotic arm.

```
class ServoNode(Node):
    def __init__(self):
        super().__init__('serve_node')
        self.pub = self.create_publisher(TwistStamped, '/servo_node/delta_twist_cmd', 10)

    def send(self, linear_x=0.0, linear_y=0.0, linear_z=0.0, angular_z=0.0, duration=1.0):
        end_time = time.time() + duration
        while time.time() < end_time:
            msg = TwistStamped()
            msg.header.stamp = self.get_clock().now().to_msg()
            msg.header.frame_id = "panda_link0"
            msg.twist.linear.x = linear_x
            msg.twist.linear.y = linear_y
            msg.twist.linear.z = linear_z
            msg.twist.angular.z = angular_z
            self.pub.publish(msg)
            time.sleep(0.02)
        self.pub.publish(TwistStamped())
```

Figure 3: ServoNode Class Overview

2

The *def __init__* defines that a publisher is created which will send movement data to the robotic arm, via the topic "delta_twist_cmd" which could be seen akin to a radio frequency, in which the frequency the node is communicating via the "delta_twist_cmd" channel.

```
1 def __init__(self):
2     super().__init__('serve_node')
3     self.pub = self.create_publisher(TwistStamped, '/servo_node/delta_twist_cmd', 10)
```

Figure 4: Initialization functions

Send can be broken into x,y,z, and duration of how long it takes for the arm to move. Using the time Python library to determine how long a directional command will go for. The "message" sent to the robotic arm is broken into the timestamp, the type of arm id, and the directionals outlines. This can be called repeatedly as desired.

```
def send(self, linear_x=0.0, linear_y=0.0, linear_z=0.0, angular_z=0.0, duration=1.0):
    end_time = time.time() + duration
    while time.time() < end_time:
        msg = TwistStamped()
        msg.header.stamp = self.get_clock().now().to_msg()
        msg.header.frame_id = "panda_link0"
        msg.twist.linear.x = linear_x
        msg.twist.linear.y = linear_y
        msg.twist.linear.z = linear_z
        msg.twist.angular.z = angular_z
        self.pub.publish(msg)
        time.sleep(0.02)
    self.pub.publish(TwistStamped())
```

Figure 5: Movement Parameters

1.1.4. Main

The rclpy.init initialises the ROS2 library is required for the script and the ROS2 to begin undertaking commands.

The *node = ServoNode()* initialises the class created with the parameters outlined and inputs.

```

3   rclpy.init()
2   node = ServeNode()
1

```

Figure 6: Starting Parameters

The following takes input and splits the variables indicated by the split comma, placing them into the function for robotic movement via x, y, z, and duration.

```

6   print("x, y, z, duration\n")
5
4   try:
3       while True:
2           cmd = input(">> ")
1           x, y, z, dur = map(float, cmd.split(','))
7
1       node.send(linear_x=x, linear_y=y, linear_z=z, duration=dur)
2

```

Figure 7: Input Loop

Once no longer required the user and press **ctrl+c** to end the input session.

The following end all node and communication sessions as to ensure no remaining messages can be sent or process remain.

```

7
6   node.destroy_node()
5   rclpy.shutdown()
4

```

Figure 8: End Loop

```

27 def main():
1   rclpy.init()
2   node = ServeNode()
3
4   print("x, y, z, duration\n")
5
6   try:
7       while True:
8           cmd = input(">> ")
9           x, y, z, dur = map(float, cmd.split(','))
10
11          node.send(linear_x=x, linear_y=y, linear_z=z, duration=dur)
12
13      except KeyboardInterrupt:
14          pass
15
16      node.destroy_node()
17      rclpy.shutdown()
18
19
20 if __name__ == '__main__':
21     main()
22

```

Figure 9: Main Loop